Multi-Tenant WhatsApp SaaS Database Architecture

Current implementation · SaaS repository · commit 15b2dc6 · 6 October 2026

CURRENT STATE: PHASE 1 FOUNDATION + PHASE 2 CATALOGUE DATA LAYER

Executive summary. The current SaaS repository implements a shared PostgreSQL tenant foundation with RLS, tenant membership, provisioning/owner protections, and a tenant-scoped products/variants catalogue. It has 18 versioned migrations and a catalogue data-access library. It does **not** contain a runnable HTTP application, a dashboard, a WhatsApp webhook runtime, or PostgreSQL operational order/conversation tables.

Tenant 001 is the development fixture `11111111-1111-4111-8111-111111111111 / cindy-bakes`. Development fixture Auth IDs, channels, WABA IDs and phone values are synthetic, not production identities or credentials.

A separate Cindy Bakes application repository contains transitional local integration: catalogue/quote reads can use PostgreSQL, while order, conversation and webhook event state remain SQLite. Those application files are not part of this SaaS repository.

Accuracy notation: solid boxes/lines are present in current migrations/code; dashed boxes/lines are separate/transitional or explicitly not implemented.

Current architecture overview

Layer	What exists	Boundary
PostgreSQL SaaS	Tenants, channels, users, settings, plans, subscriptions, audit log, products, variants; helper functions; migration ledger.	RLS enforces tenant scope. Migration ledger is metadata, not tenant data; migrations do not configure its RLS.
Separate Cindy app SQLite	orders, whatsapp_conversations, whatsapp_events, invoice/payment workflow.	Not in SaaS repo and not migrated. Catalogue IDs are copied as text; no cross-database FK.
Runtime/Auth	SaaS Python provides DB connection, tenant context, routing helper, provisioning function, catalogue service.	No HTTP/API/Auth gateway/dashboard runtime in SaaS repo. JWT signature verification is upstream, not implemented by SQL helper.

MULTI-TENANT SaaS DATABASE ARCHITECTURE

Current implementation at 15b2dc6 | PostgreSQL migrations 0001-0018 | SQLite boundary shown separately

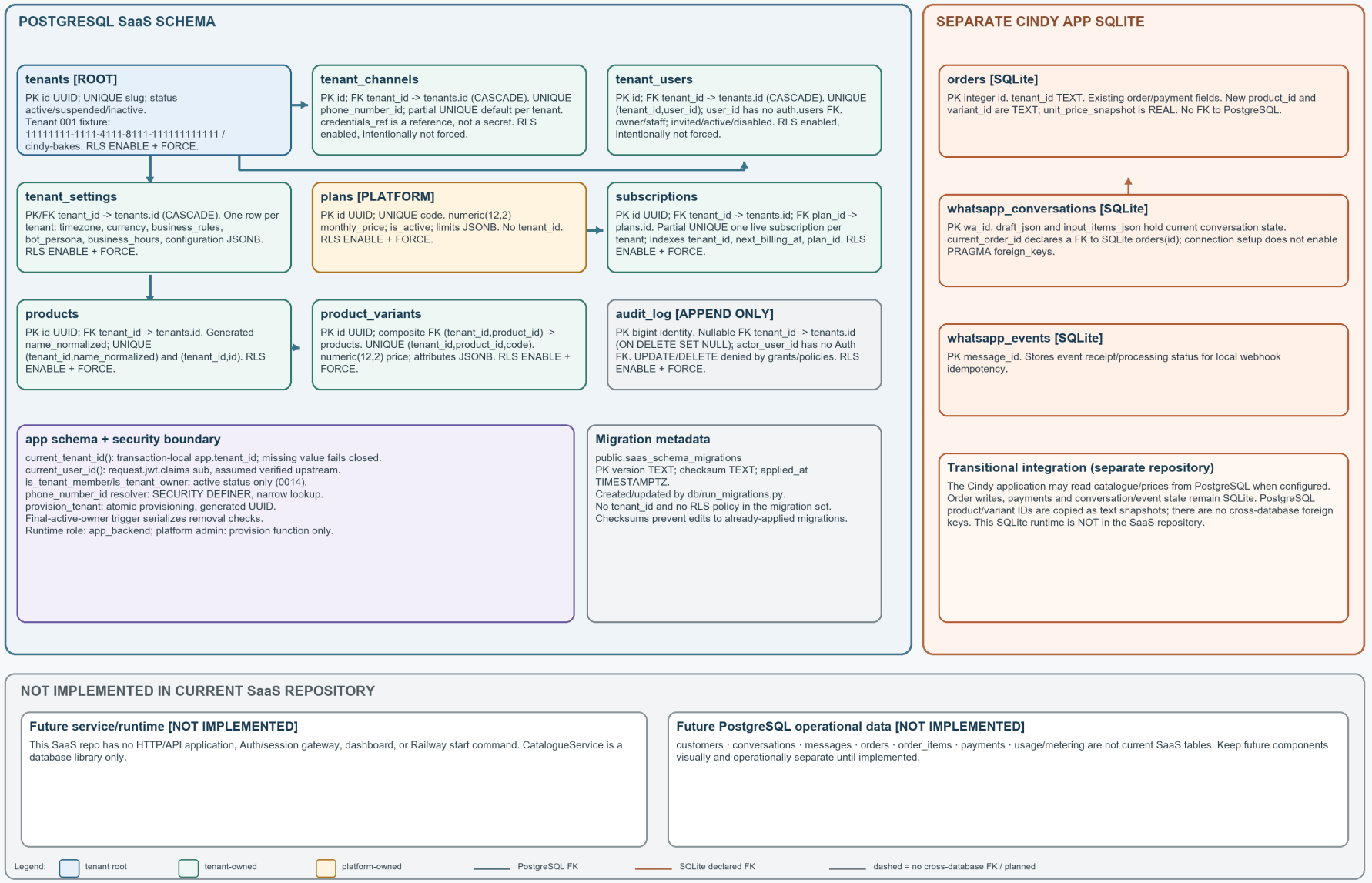


Table-by-table data dictionary

PostgreSQL domain tables are created by migrations 0003–0009 and 0017; app-schema helpers are functions, not tables. RLS summaries reflect migrations 0011–0018.

Table	Purpose / PK / important columns	FKs / unique constraints / indexes	Tenant relationship / RLS
tenants	Root business identity. PK id UUID. Columns: name, slug, status, timestamps.	slug UNIQUE; slug/name/status checks; status index. No tenant_id (id is the tenant identity).	Root. Backend reads/updates current tenant; Authenticated active member reads and active owner updates. ENABLE + FORCE RLS.
tenant_channels	Per-tenant WhatsApp channel record. PK id UUID. Columns include channel_type, phone_number_id, waba_id, display_phone_number, status, is_default, credentials_ref.	FK tenant_id → tenants.id ON DELETE CASCADE; phone_number_id UNIQUE globally; partial UNIQUE one default/tenant; tenant index. credentials_ref is a reference, not a token.	Tenant-owned. Backend all commands in context; active member SELECT; active owner writes. RLS enabled, not forced for SECURITY DEFINER routing lookup.
tenant_users	Tenant membership. PK id UUID. Columns: tenant_id, user_id UUID, role, status, timestamps.	FK tenant_id → tenants.id ON DELETE CASCADE; UNIQUE (tenant_id,user_id); indexes user_id and tenant_id. No FK to auth.users.	Tenant-owned. Self/owner SELECT; active owner writes; helper authorization requires status=active. RLS enabled, not forced for SECURITY DEFINER membership helpers.
tenant_settings	One configuration row per tenant. PK/FK tenant_id. Columns: timezone, currency, business_rules, bot_persona, business_hours, configuration, timestamps.	FK tenant_id → tenants.id ON DELETE CASCADE. JSONB object checks, ISO currency check.	Tenant-owned 1:1. Backend scoped read/write; active members read; active owners write. ENABLE + FORCE RLS.
plans	Platform subscription plan catalogue. PK id UUID. Columns: code, name, description, monthly_price numeric(12,2), currency, is_active, limits JSONB, timestamps.	code UNIQUE; code/price/currency/JSON checks; is_active index. No tenant_id.	Platform-owned. Backend reads active and inactive (historical subscriptions); authenticated users read active only. ENABLE + FORCE RLS; no app writes.
subscriptions	Tenant subscription state. PK id UUID. Columns: tenant_id, plan_id, status, billing_method, period/trial/next/cancel dates, timestamps.	FK tenant_id → tenants.id ON DELETE CASCADE; FK plan_id → plans.id. Partial UNIQUE one live row/tenant; indexes tenant_id, next_billing_at, plan_id.	Tenant-owned. Backend scoped SELECT/INSERT/UPDATE; active members SELECT. ENABLE + FORCE RLS. Manual subscription state exists; automated billing does not.
audit_log	Append-only event history. PK id BIGINT identity. Columns: nullable tenant_id, actor_user_id, actor_type, action, entity_type/id, metadata JSONB, created_at.	Nullable FK tenant_id → tenants.id ON DELETE SET NULL. actor_user_id has no Auth FK. Indexes tenant/created, entity, action.	Tenant/platform events. Backend inserts/reads own tenant events; active members read own tenant. Platform NULL rows hidden. No UPDATE/DELETE policy/grant. ENABLE + FORCE RLS.
products	Tenant catalogue item. PK id UUID. Columns: tenant_id, name, generated name_normalized, description, category, is_active, sort_order, attributes JSONB, timestamps.	FK tenant_id → tenants.id ON DELETE CASCADE; UNIQUE (tenant_id,name_normalized), UNIQUE (tenant_id,id); check constraints; tenant/active/sort index.	Tenant-owned. Backend full DML in context; active members SELECT; active owners DML. ENABLE + FORCE RLS.
product_variants	Priced option under a product. PK id UUID. Columns: tenant_id, product_id, code, name, price numeric(12,2), attributes JSONB, is_active, sort_order, timestamps.	Composite FK (tenant_id,product_id) → products(tenant_id,id) ON DELETE CASCADE; UNIQUE (tenant_id,product_id,code); check constraints; tenant/product/active/sort index.	Tenant-owned, parent tenant preserved by composite FK. Same backend/member/owner policy split as products. ENABLE + FORCE RLS.
saas_schema_migrations	Migration runner metadata table. PK version TEXT. Columns: checksum, applied_at.	No domain FKs, no tenant relationship. SHA-256-derived checksum recorded by runner.	Not tenant data. Created in public by runner; migration set does not define RLS/grants for it; access follows creator/default grants.

SQLite operational tables in the separate Cindy Bakes application repository

Table	Columns / purpose	Relationships / boundary
orders	PK integer id. Includes tenant_id TEXT, existing customer/order/payment/delivery fields, product_id/variant_id TEXT and unit_price_snapshot REAL.	SQLite only. No PostgreSQL FK. Schema columns are added lazily by the app. tenant_id defaults to Tenant 001 fixture ID.
whatsapp_conversations	PK wa_id TEXT. phone_number, draft_json, input_items_json, current_order_id, activity timestamps.	Declares SQLite FK current_order_id → orders.id. The app connection does not enable PRAGMA foreign_keys, so enforcement is not active by default.
whatsapp_events	PK message_id TEXT. wa_id, message_type, received/processed timestamps, processing_status, error_message.	SQLite-only webhook idempotency/event state. Not the planned PostgreSQL webhook_events table.

Tenant isolation model

Tenant-owned tables use tenant_id; tenants is the root (id is tenant identity); plans is platform-owned; audit_log permits NULL tenant_id only for platform events. RLS is enabled for all domain tables. FORCE RLS is applied except tenant_channels and tenant_users, which SECURITY DEFINER helpers read; application-role policies still filter both.

app_backend policies compare tenant_id with app.current_tenant_id(), which reads a transaction-local setting. Missing context returns NULL and fails closed. Authenticated policies call membership/owner helpers. Product variants use a composite tenant-preserving FK. Tests inspect the tenant_id inventory and assert cross-tenant reads/writes fail.

Authentication and membership model

app.current_user_id() reads request.jwt.claims sub. SQL migrations do not verify JWT signatures; a trusted PostgREST/Auth gateway must do so. tenant_users.user_id is intended to be the verified identity UUID but deliberately has no FK to auth.users.

Migration 0014 defines only status=active as member/owner authorization. Invited, disabled, and deleted memberships do not authorize. A trigger prevents removal/demotion of the final active owner. The provisioner function atomically creates a tenant, active owner, settings, optional subscription, and audit record; it generates a UUID rather than accepting a fixed ID.

Catalogue architecture

products has a generated normalized name and tenant-scoped uniqueness. product_variants stores active priced options with numeric(12,2) prices and JSONB attributes. Its composite FK preserves tenant identity. Both tables have indexes beginning with tenant identity, RLS ENABLE + FORCE, backend contextual policies, member reads, and owner writes.

saas.catalogue.CatalogueService requires an explicit tenant ID and performs reads in tenant_transaction. It lists active products/variants, resolves normalized names, and resolves weight variants. The Cindy development catalogue seed looks up slug cindy-bakes; it creates no tenant and is idempotent. Local validation found 11 products and 33 variants, but these are fixture data.

Order and operational data architecture

No PostgreSQL customers, conversations, messages, orders, order_items, payments, or usage tables exist in this branch. They are described in DATABASE_PLAN.md as future design, not implemented schema.

The separate Cindy app repository has transitional catalogue reads/quotes using PostgreSQL when SAAS_DATABASE_URL is set, while confirmed orders, invoices, conversation drafts, and webhook event state remain SQLite. SQLite product/variant IDs are TEXT snapshots/references and unit_price_snapshot is REAL; no cross-database FK or atomic transaction exists.

PostgreSQL vs SQLite boundary

PostgreSQL is current SaaS storage for tenancy, channels, memberships, settings, plans, subscriptions, audit, products, and variants. SQLite operations are owned by the separate Cindy app repository and are not part of the SaaS database or this branch.

The Cindy SQLite schema declares a conversation current_order_id FK to orders.id, but its connection code does not enable PRAGMA foreign_keys. This is a declared but not actively enforced SQLite relation. PostgreSQL does not reference SQLite records.

Migration history 0001–0018

0001 extensions/app schema/context helpers; 0002 roles; 0003 tenants; 0004 channels; 0005 memberships; 0006 settings; 0007 plans; 0008 subscriptions; 0009 append-only audit log; 0010 membership and phone routing helpers; 0011 RLS; 0012 grants; 0013 platform provisioning, owner trigger, plan lookup index/default sequence grants; 0014 active-only membership and final-owner locking; 0015 platform provisioning login; 0016 provisioner public-schema usage; 0017 products/variants; 0018 catalogue RLS/grants.

db/run_migrations.py sorts migrations by filename and records checksums. Applied migrations are immutable; changed checksums stop the runner. The development seed runner sorts files by filename: 0001_dev_seed before 0002_cindy_bakes_catalogue.

Cindy Bakes / Tenant 001

The deterministic development fixture uses UUID 11111111-1111-4111-8111-111111111111 and slug cindy-bakes. Its Auth user IDs and fake channel identifiers are synthetic. They are not production owner identity or credentials.

The development base seed also inserts Test Bakery, fake channels, fixture memberships, placeholder settings, plan/subscriptions, and audit rows. The catalogue-only seed inserts Cindy products/variants only if cindy-bakes exists. The production-safe bootstrap path must first verify ID/slug and use a real staging Auth UUID. app.provision_tenant generates a UUID, so it cannot itself satisfy a fixed Tenant 001 UUID requirement.

Current security controls

app_backend is a non-owner, non-BYPASSRLS role. The SaaS connection now verifies current_user and session_user are app_backend, rejects superuser/BYPASSRLS, public table ownership, and elevated/service-role membership, and closes the connection on rejection.

RLS has tenant-context USING/WITH CHECK policies; authenticated access uses active membership; variants have a composite FK; audit_log has no update/delete grants or policies; provisioning is a restricted SECURITY DEFINER function; owner removal is guarded; tenant context is transaction-local. The runner has checksum protection and localhost/confirmation guards for --reset.

SECURITY DEFINER helpers use fixed search_path and narrow EXECUTE grants. tenant_channels and tenant_users remain the intentional non-FORCE exceptions; test coverage verifies their policies for app roles.

Current limitations / transitional architecture

The SaaS repo contains no Flask service, HTTP routes, Auth gateway, SaaS dashboard, WhatsApp webhook, worker, or Railway start command. It is not independently runnable as an HTTP SaaS application today.

The separate Cindy app runtime can use the PostgreSQL catalogue but still writes operational orders/conversations in SQLite. Enabling SAAS_DATABASE_URL there affects browser and WhatsApp agent catalogue/quote reads; database failures can fail a claimed WhatsApp event. This runtime is not in the SaaS branch.

Tenant settings currently do not drive Cindy app business rules; those remain application code. The app's SQLite snapshot data is not PostgreSQL-normalized and must be migrated/reconciled in a future cutover.

Recommended future architecture (proposed, not implemented)

Add an explicit SaaS HTTP/API service in this repository, integrate a trusted Auth issuer and tenant selection, and make every request enter through app_backend tenant transactions. Keep provisioning behind its platform capability.

For operational PostgreSQL data, add versioned tenant-scoped customers/conversations/messages/orders/order_items/payments migrations with tenant-preserving FKs, RLS, policies, indexes, and structural tests. Plan a rehearsed SQLite migration that preserves snapshots, order IDs/mapping, payment verification history, conversations, webhook idempotency, and rollback before live cutover.

Add backup/restore drills, monitoring, retention, DPA export/deletion workflows, and environment-specific migration/seed tooling before onboarding real tenants.

Deployment considerations

This document is descriptive, not a deployment runbook. The current SaaS repository has no Railway start command. Build a runnable service separately, use separate staging/production databases and roles, verify migration checksums, and keep runtime credentials on the server only.

Never run --seed in a production-like database: 0001_dev_seed adds Test Bakery and fake channels. Provision and verify the staging tenant/owner separately; apply only the catalogue seed after verifying the tenant. Do not mix similarly numbered migration sets from another repository.

PostgreSQL schema migrations do not apply to or modify the separate SQLite database. Do not connect the live Cindy WhatsApp number or existing Railway service to this schema until the SaaS runtime and cutover are separately approved.